

Unidade I

1 FUNDAMENTOS DE C#

C# é uma linguagem de programação criada pela empresa Microsoft. Ela serve para construir muitos tipos de software: sites, serviços em nuvem, programas de computador, aplicativos para celular e jogos. É moderna, com checagem de tipos (ajuda a evitar erros) e gerenciamento automático de memória, além de ter recursos de produtividade como `async/await` e LINQ. C# é a linguagem principal da plataforma .NET.

.NET é a plataforma que executa programas escritos em C#. A edição .NET 10 é o lançamento de 2025 (ciclo anual da Microsoft) e é um LTS (Long Term Support)

1.1 Sintaxe, tipos, nullability, var, organização de solução/projeto

A linguagem C# pertence à família das linguagens derivadas de C, possuindo uma sintaxe bastante familiar para quem já utilizou C, C++, Java ou JavaScript. Nela cada instrução termina com ponto e vírgula (;) e blocos de código são delimitados por chaves { e }. Os identificadores (nomes de variáveis, métodos, classes etc.) são case-sensitive, ou seja, diferencia maiúsculas de minúsculas, inclui as estruturas de controle usuais, como `if/else` para condicionais, `switch` para seleção de casos e laços de repetição `for`, `while` e `foreach`, com sintaxes muito semelhantes às do C/C++.

No código tradicional, começamos declarando um namespace (Exemplo) para organizar logicamente a classe Program. Dentro da classe, definimos o método Main com o modificador `static`, indicando que é um método de classe (e não de instância). O método Main sem parâmetros é convencionalmente o ponto de entrada de programas C# executáveis – ou seja, é a primeira função a ser invocada quando o programa é iniciado. Nele, fazemos a chamada `Console.WriteLine` para produzir saída no console.

C# é uma linguagem fortemente tipada, o que significa que cada variável e constante no código possui um tipo bem definido em tempo de compilação. Toda expressão em C# avalia para um tipo específico, conhecido pelo compilador, e operações envolvendo valores geralmente só são permitidas se os operandos forem de tipos compatíveis. Esse forte sistema de tipos promove a detecção precoce de erros: o compilador (ou mesmo as ferramentas do ambiente de desenvolvimento) emite erros ou avisos caso tentemos usar uma variável de modo inconsistente com seu tipo declarado.

Algumas dessas palavras-chaves são `int` (em C#), que corresponde ao tipo inteiro de 32 bits assinado (`System.Int32` na plataforma .NET); da mesma forma, `double` é um tipo de ponto flutuante de 64 bits (`System.Double`); `char` representa um caractere Unicode de 16 bits (`System.Char`); `bool` é um valor booleano lógico (`System.Boolean`, que pode ser `true` ou `false`), e assim por diante. O quadro 1 ilustra alguns dos tipos primitivos e seus propósitos.

Tipos por valor são aqueles que armazenam diretamente seus dados. Isso significa que uma **variável de um tipo por valor contém em si o valor atribuído**, e não uma referência a um valor em outro lugar. Os **tipos numéricos primitivos (int, double etc.), bool, char e também os tipos definidos com struct (estruturas) são tipos por valor.** Por padrão, instâncias de tipos por valor são **alocadas na stack (pilha)** de execução ou embutidas dentro de estruturas de dados maiores. Uma consequência importante é que, ao atribuir uma variável de tipo por valor a outra, realiza-se uma cópia dos dados.

Tipos por referência, por outro lado, armazenam referências (endereços) para objetos alocados na heap (área de memória dinâmica). Class (classes), arrays (vetores) e string (cadeia de caracteres) são exemplos de tipos por referência. Quando declaramos uma variável de um tipo por referência, **inicialmente ela não aponta para nenhum objeto concreto e seu valor padrão é null (nulo), indicando nenhuma referência válida.** Para obter um objeto real, geralmente usamos o operador **new** para instanciar a classe, o que aloca memória na heap e retorna uma referência para essa região de memória. A variável então armazena esse ponteiro (de forma segura, sem aritmética de ponteiros explícita na linguagem gerenciada). **Ao atribuir uma variável de referência a outra, não copiamos os dados do objeto em si, mas sim a referência (o endereço).**

Obs: Instanciar a classe significa criar um objeto concreto a partir de uma classe

Outra característica dos tipos por referência é que eles podem representar a ausência de um valor real através de null. Uma variável de referência não inicializada (ou explicitamente setada como null) indica que ela não aponta para nenhum objeto válido naquele momento; tentar usar (desreferenciar) uma referência nula em tempo de execução causa a exceção `System.NullReferenceException`

Em memória de programas, **stack e heap** descrevem **estratégias distintas de organizar dados** ao longo da execução. **A primeira se assemelha a uma pilha** de pratos: tudo que entra por cima sai por cima, em ordem estritamente **last-in, first-out (LIFO)**. **A segunda lembra um almoxarifado: há espaço para itens de tamanhos variados, guardados e recuperados conforme a necessidade**, sem seguir a mesma rigidez de ordem.

A pilha serve de suporte imediato à execução de funções. Quando um procedimento começa, o sistema cria um quadro de ativação que costuma reunir parâmetros, variáveis locais e o endereço de retorno; ao término, esse quadro é descartado integralmente. Como contrapartida, o espaço é limitado e o padrão LIFO impõe vidas curtas e bem delimitadas aos dados; empilhar estruturas grandes ou encadear chamadas profundas pode resultar em estouro de pilha, e dados com tamanho muito variável ou que precisam persistir além da execução da função não são adequados para esse tipo de armazenamento.

O heap, por sua vez, atende a dados cujo tamanho ou duração não cabem nas amarras da pilha. Estruturas dinâmicas, coleções que crescem com a entrada do usuário e objetos que atravessam fronteiras de funções são alocados nessa área. Em ambientes com GC, como a plataforma .NET, o runtime monitora a alcançabilidade dos objetos e libera a memória quando deixam de ser referenciados; em linguagens sem coleta automática, cabe ao programador desalocar no momento correto. A flexibilidade tem custo: a alocação exige

metadados e coordenação do alocador, e a liberação depende do coletor ou de disciplina manual. Além disso, referências mantidas desnecessariamente podem causar retenção de memória

A pilha privilegia velocidade e previsibilidade ao preço de limites rígidos; o heap oferece maleabilidade com sobrecusto de gerenciamento.

Tipos por valor são eficientes e evitam alocação dinâmica quando contêm dados pequenos e imutáveis, mas não podem ser nulos por padrão. Tipos por referência acomodam estruturas de dados mais complexas e de tamanho variável, permitindo compartilhamento de objetos, ao custo de ser preciso gerenciar (indiretamente) o fato de que múltiplas referências podem apontar para o mesmo objeto.

Comparações de igualdade (`==` em tipos por valor compara conteúdo, enquanto em tipos por referência compara referência, exceto se o operador for sobrecarregado) ou questões de gerenciamento de memória

Um exemplo como `int x = 5;` ilustra o caso comum de um valor simples associado ao escopo imediato da função: variáveis locais de tipos por valor tendem a residir na pilha durante a execução, o que favorece acesso rápido e descarte automático ao final do bloco. A documentação apresenta justamente essa intuição ao discutir como reduzir alocações ao preferir **struct** (um tipo de valor que pode encapsular dados e funcionalidades relacionadas.) em cenários adequados.

Quando a instrução envolve `new`, como em `var pessoa = new Pessoa();`, a instância resultante é criada no heap gerenciado, enquanto a variável local retém apenas uma referência a esse objeto. Essa separação entre a variável que contém o valor (tipo por valor) e a variável que contém uma referência (tipo por referência) é um ponto central da especificação da linguagem.

No ecossistema .NET, a memória do heap é administrada pelo GC. O runtime acompanha a alcançabilidade dos objetos e, quando não existem mais referências ativas, sua memória se torna elegível para liberação; o processo é automático e poupa o desenvolvedor de desalocação manual, evitando erros clássicos de uso após liberação. Para casos estritamente temporários e com tamanho conhecido, a linguagem oferece ainda `stackalloc`, que reserva um bloco na pilha e o descarta integralmente ao retorno do método, fora do escopo do GC.

Os tipos anuláveis por valor (nullable value types). Usando a sintaxe de interrogação após o tipo – por exemplo, `int?` em vez de `int` –, podemos declarar um tipo por valor que aceite representar também a ausência de valor através de `null`. Em termos técnicos, `int?` é um complemento sintático para `System.Nullable`, uma estrutura genérica que encapsula um valor do tipo especificado e um flag indicando se há um valor presente ou não. Assim, `int? x = null;` é válido e inicializa `x` sem um valor inteiro concreto.

Esse recurso foi criado para lidar com situações como campos de banco de dados ou informações opcionais que possam ser desconhecidas ou indisponíveis, aplicando-se apenas a tipos por valor, já que tipos por referência sempre puderam ser nulos.

Se tentamos atribuir null a uma variável que foi declarada como de tipo não anulável (por exemplo, fazer nome2 = null onde nome2 é string não anulável), o compilador emite um aviso ou um erro conforme a configuração, pois estamos violando a promessa de não nulidade, se uma variável é não anulável, o compilador espera que ela seja inicializada com um valor não nulo antes de ser usada.

Para suprimir intencionalmente um aviso desses (quando se tem certeza do estado não nulo), C# fornece o operador de null-forgiveness !. Por exemplo, poderíamos escrever string texto = mensagem!; para indicar ao compilador: “confie em mim, mensagem não é nula aqui”. Isso remove o aviso, porém deve ser usado com cautela: é uma forma de sobrescrever o mecanismo de análise estática e, se usado incorretamente (quando a variável for de fato nula), o erro de runtime continuará existindo. Idealmente, deve-se preferir checagens explícitas de null (if (mensagem != null) { ... }) antes de utilizar a variável, ou empregar o operador de coalescência nula (??) para fornecer valores alternativos em caso de null. Nos bastidores, string e string? são do mesmo tipo System.String no runtime, e a distinção existe apenas para o compilador.

A nulabilidade no C# moderno envolve duas frentes: (1) os tipos por valor anuláveis (T?, em que T é um valor, via Nullable), que existem desde C# 2.0 e permitem representar null para structs e primitivos; e (2) os tipos por referência anuláveis (habilitados em C# 8+), que introduzem análise estática para evitar usos inadvertidos de referência nula.

Em C#, declaramos variáveis especificando primeiramente o tipo, seguido do nome da variável e, opcionalmente, de uma atribuição inicial.

Var só pode ser utilizado em variáveis locais (dentro de métodos ou blocos de código). Ele não é permitido em campos de classe ou estrutura, nem em parâmetros de métodos.

Use var quando o tipo do lado direito é óbvio para um leitor humano ou quando é muito verboso. Por exemplo, ao escrever var lista = new List();, é bastante claro que lista será do tipo List, eliminando a repetição do tipo em ambas as partes da declaração. Por outro lado, em algo como var numero = ObterValor();, no qual o tipo retornado por ObterValor() pode não ser evidente, talvez seja melhor especificar o tipo explicitamente para melhorar a legibilidade, a menos que o contexto deixe claro do que se trata.

Em situações de tipos anônimos, var é obrigatório. O C# permite criar objetos ad hoc com propriedades sem definir uma classe formal – os chamados tipos anônimos. Por exemplo: var anon = new { Nome = “Maria”, Idade = 30 };. Aqui, anon terá um tipo anônimo gerado pelo compilador (sem nome acessível no código), então não haveria como declarar essa variável de outra forma senão com var. Em consultas LINQ, por exemplo, muitas vezes obtemos sequências de objetos anônimos, e nesses casos usar var é necessário para manipular os resultados.

Exemplificando o cenário de tipos anônimos e var em conjunto, veja este trecho:

```
var resultado = from c in clientes where c.Cidade == "São Paulo" select new {  
    NomeCliente = c.Nome, AnoNascimento = c.DataNascimento.Year };  
  
foreach (var item in resultado) {  
    Console.WriteLine($"Cliente: {item.NomeCliente}, Nascido em: {item.AnoNascimento}");  
}
```

Em C# no ecossistema .NET, o código-fonte é escrito dentro de projetos, e esses projetos são agrupados em uma solução. A solução (arquivo .sln) é um contêiner lógico: ela guarda a lista de projetos, as dependências entre eles, as configurações de build e os perfis de execução. Cada projeto (arquivo .csproj) define como compilar um artefato (uma biblioteca, um aplicativo de console, uma Web API, um conjunto de testes). Essa separação permite evoluir módulos de forma independente e reutilizar bibliotecas em diferentes aplicativos.

Dito de outro modo, o código é organizado em projetos (.csproj) que geram artefatos específicos – bibliotecas reutilizáveis, aplicativos de console, serviços web – e em uma solução (.sln) que agrega esses projetos, registra as dependências entre eles e guarda configurações de compilação e execução. A solução funciona como índice da aplicação e dita a ordem de build com base nas referências declaradas; cada projeto, por sua vez, define o que compilar, com quais opções e quais outras unidades de código consome.

1.2 Ferramentas: Visual Studio 2026 e dotnet CLI (estrutura .csproj, SDK-style)

parei na 30